

ARITHMETIC OPERATIONS WITH POINTERS

```
#include <stdio.h>
```

```
int main() {
```

```
    int x=5;
```

```
    int *p=&x;
```

```
    *p=x;
```

```
    printf("address of x=%p\n", p); → screen; 62FE14
```

```
    p++; // 4 byte address
```

```
    printf("address of x=%p\n", → screen; 62FE18
```

```
    p+=2; // 8 byte address
```

```
    p=p-2; // 8 byte address
```

```
void swap(int x, int y) {
```

```
    int temp;
```

```
    temp=x;
```

```
    x=y;
```

```
    y=temp;
```

```
    printf("x=%d, y=%d\n", x, y) → screen x=10 y=5
```

```
int main() {
```

```
    int x=5, y=10;
```

```
    swap(x, y);
```

```
    printf("x=%d, y=%d\n", x, y); → screen x=5 y=10
```

```
    return 0;
```

```
void swap(int *x, int *y) {
```

```
    int temp;
```

```
    temp=*x
```

```
    *x=*y
```

```
    *y=temp;
```

```
int main() {
```

```
    int x=5, y=10;
```

```
    swap(x, y);
```

```
    printf("x=%d, y=%d\n", x, y);
```

```
    → screen x=10, y=5
```

Not: Bir fonksiyonu birden fazla deger döndürülebilmesi için pointer kullanılır

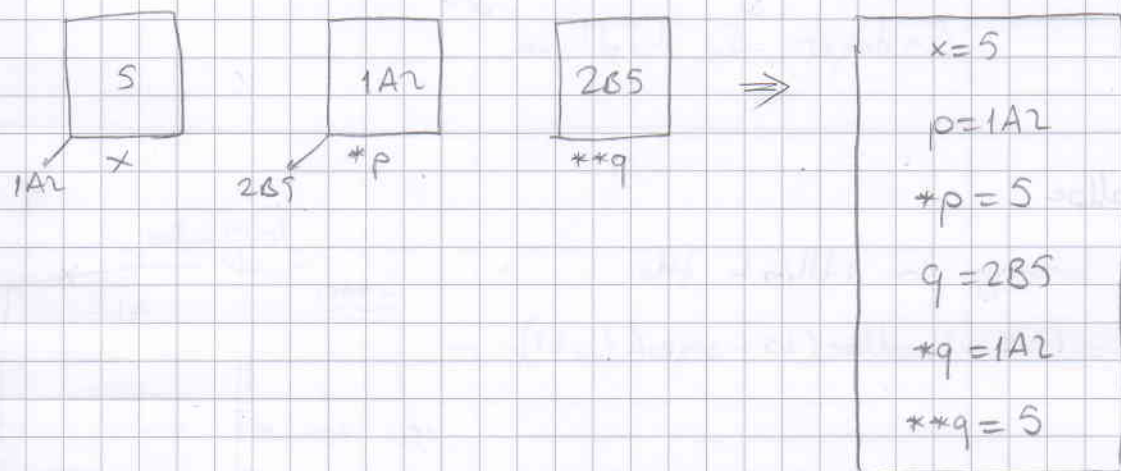
POINTERS TO POINTERS

```
int x=5;
```

```
int *p=&x;
```

```
int **q=&p;
```

$**q \rightarrow q$ is a pointer to pointer and it keeps and address of a pointer.



ARRAYS AND POINTERS

We can use pointers to define dynamic arrays.

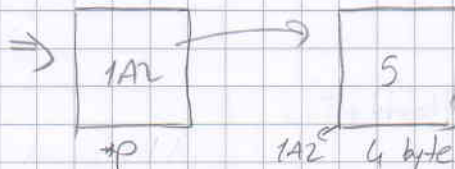
```
#include <stdlib.h>
```

```
*p;
```

```
p=(int*) malloc(4);
```

↳ the area to be allocated in terms of bytes;

```
*p=5;
```



```
int x=(int) 3.2 => x=3
```

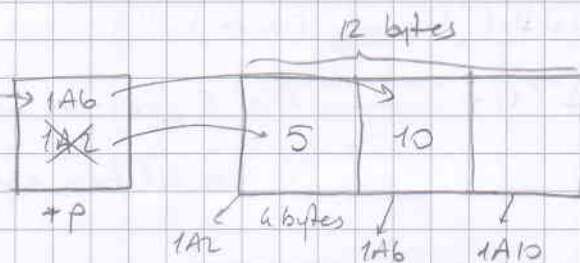
```
float x=(float) 4 => x=4.0
```

```
p=(int*) malloc(12)
```

```
*p=5
```

```
p++;
```

```
*p=10;
```




```

int A[5], i;
for (i=0, i<5, i++) {
    A[i]=i; // *(A+i)=i;
    printf("%d\n", A[i]);
}

```

```

#include <stdlib.h>
int i;
int *p;
p = (int *) malloc (5 * sizeof (int));
for (i=0, i<5, i++) {
    *(p+i)=i // p[i]=i;
    printf("%d\n", *(p+i));
}

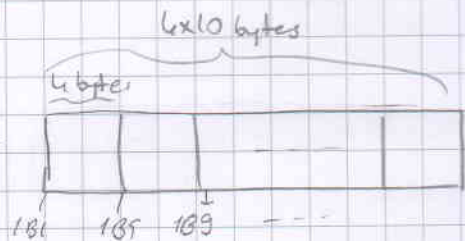
```

Esdeger iki kodlamo

Malloc

It belongs to stdlib.h file

$p = (\text{int} *) \text{malloc}(10 + \text{sizeof}(\text{int}));$ →



If there is no sufficient space, malloc returns NULL.

$\text{NULL} = 0$

We use NULL to assign zero to the pointers.

$\text{int} *p = \text{NULL};$ // p does not contain any addresses.

When we call malloc function, we should check whether the disk has enough space.

$\text{float} *fp;$

$fp = (\text{float} *) \text{malloc}(\text{sizeof}(\text{float}) * 5);$

if ($fp == \text{NULL}$) {

printf("Memory Error\n");

exit(1); → the C program will be terminated

} → It has not been completed successfully